人工智能（大模型）-笔试题

时间：2026 年 07 月 13 日 9:00-07 月 17 日 17:00

基于 AI Agent 的算法能力系统开发

背景

随着大语言模型和 AI Agent 技术的发展，算法开发正在从“人工理解需求—手工编写代码—人工测试部署”的模式，逐步转向“需求理解—能力抽取—算法复刻—自动验证—能力沉淀”的智能化模式。

在真实行业场景中，大量算法能力往往分散存在于业务文档、历史代码仓库、专家经验、实验记录和测试报告中。这些能力包括数据预处理方法、模型选择经验、特征工程策略、评价指标、失败案例、调参经验和业务约束等。如果能够将这些能力自动抽取并结构化沉淀，再通过 AI Agent 进行算法复刻和自动验证，将有助于构建一套“基于 AI Agent 的算法能力工厂”平台，实现从业务需求或能力描述到可用算法代码的端到端半自主/自主生成。

本题要求候选人设计并实现一个小型原型系统，验证“能力抽取—能力复刻—能力验证与沉淀”的核心流程。

任务

请设计并实现一个基于 AI Agent 的算法能力工厂原型系统，要求：

1. 使用开源大语言模型或可调用的 LLM 接口，完成算法能力理解、知识抽取、

代码生成或代码修复；

2. 选择一个具体行业场景，例如客户流失预测、异常检测、库存预测、商品推荐、文本分类、图像缺陷识别等；

3. 构建一个小型行业算法能力知识库或知识图谱，用于表达算法能力、输入输出、适用条件、评价指标、依赖环境和历史验证结果；

4. 实现从自然语言能力描述到可运行算法代码的自动或半自动复刻流程；

5. 建立统一验证机制，对生成算法进行功能正确性、指标表现、运行稳定性和接口规范检查；

6. 提供简洁的用户界面、CLI 或 API，使用户可以提交能力描述并获得生成结果。

系统应体现以下完整流程：

a. 理解用户输入的算法能力描述；

b. 从文档、代码示例或知识图谱中检索相关能力知识；

c. 规划算法实现方案；

d. 生成可运行代码；

e. 自动执行测试与指标评估；

f. 根据错误信息进行修复或优化；

g. 将验证结果沉淀回知识库或知识图谱。

开发要求

a. 使用 Git 进行版本控制，提交信息清晰；

b. 提供完整项目文档，包括项目说明、环境配置、运行指南和示例；

c. 代码结构清晰，模块化设计，注释完整；

d. 代码应能够在本地环境复现运行；

e. 建议使用 Python 作为主要开发语言；

f. 可使用 LangChain、LlamaIndex、AutoGen、CrewAI、OpenAI API、本地开源模型、Neo4j、NetworkX、SQLite、FastAPI、Streamlit 等工具，但需说明选择理由；

g. 若无法部署大模型，可使用 mock LLM、规则模板或 API 调用方式完成核心流程演示。

交付内容

基础要求

- 完整的项目代码库；
- 示例行业数据、文档或模拟业务材料；
- 能力知识图谱 schema 设计说明；
- 知识抽取结果文件，例如 JSON、CSV、RDF、GraphML 或 Neo4j 导入文件；

- Agent 工作流代码或流程配置；
- 至少 1 个从能力描述到算法代码生成的完整示例；
- 自动验证脚本；
- 验证报告样例；
- 项目文档 README.md；
- 环境依赖文件 requirements.txt 或 environment.yml；
- 使用示例和测试用例。

进阶要求

- 提供 Web 界面、CLI 或 API 接口；
- 支持多轮代码修复；
- 支持多个候选算法方案自动比较；
- 支持知识图谱可视化；
- 支持自动生成验证报告；
- 支持将验证结果回写到知识库或知识图谱；
- 支持不同任务类型的验证配置；
- 支持插件式接入新的算法模板或评估指标。

加分项

- 引入图搜索、Beam Search 或 MCTS 进行算法方案搜索；
- 实现多智能体协作，而不是单次 prompt 生成代码；
- 支持从真实代码仓库中抽取函数、依赖和能力描述；
- 支持代码安全检查和沙箱执行；
- 支持自动分析失败案例，并形成可复用经验；
- 支持算法能力的版本管理；
- 支持自然语言解释生成方案的设计依据；
- 支持跨场景迁移，例如同一套框架可用于表格分类和文本分类；
- 支持自动生成接口文档或部署配置；
- 对生成算法进行性能优化和资源消耗分析。

评估标准

技术能力（40%）

- 是否能够完整实现“能力抽取—能力复刻—能力验证—能力沉淀”的核心闭环；
- 对 LLM Agent、代码生成、知识图谱和算法验证的理解深度；
- 知识图谱 schema 设计是否合理；
- Agent 工作流是否清晰有效；
- 自动验证机制是否可执行、可扩展；
- 是否具备代码修复、候选方案比较或搜索优化能力。

代码质量（30%）

- 代码规范性和可读性；
- 模块设计是否清晰；
- 接口定义是否合理；
- 错误处理和日志记录是否完善；
- 项目是否容易运行和复现；
- 是否有测试用例；
- 是否避免硬编码和不可维护设计。

创新性（15%）

- 是否提出有创造性的 Agent 协作机制；
- 是否有效利用知识图谱增强代码生成和验证；
- 是否设计了合理的搜索、优化或自修复策略；
- 是否能将失败经验沉淀为可复用知识；
- 是否考虑真实行业落地中的复杂问题。

完整性（15%）

- 功能完成度；
- 文档质量；
- 示例是否完整；
- 验证报告是否清晰；
- 用户体验是否友好；
- 是否能够展示端到端运行结果。

提交方式

本题为全开放题，可自由选用各类工具完成。

请于 7 月 17 日 17:00 前将成果打包为 zip 压缩包后，发送至邮箱：

sijiachen@intl.zju.edu.cn。

同时，需要在 README 中明确说明：

1. 项目背景和目标；
2. 系统架构和模块设计；
3. 能力知识图谱 schema 和示例；
4. Agent 工作流设计；
5. 环境配置和运行方法；
6. 示例数据和测试任务说明；
7. 生成算法代码示例；
8. 验证结果和报告样例；
9. 遇到的挑战和解决方案；
10. 后续可扩展方向。